BRAC UNIVERSITY
Department of Computer Science and Engineering

A

Examination: Mid-Term
Duration: 75 Minutes
Number of Questions: 4

CSE220: Data Structures

Summer 2025
Full Marks: 30
No. of Pages: 3

Name: (Please write in CAPITAL LETTERS)	ID:	Section:

- **Answer all 4 questions. No washroom breaks.**
- At the end of the exam, put the question **paper** inside the answer script and **return both**.

Question 1: CO1 [8 Points]

You're working as a terrain analyst for a Mars exploration rover. The rover receives a grid (matrix) of terrain height values scanned from orbit. However, due to occasional solar interference, some readings show spikes (noisy values) that are unrealistically high and should be ignored.

Your task is to complete the following function/method to help the rover find a local minimum — a position where the height is lower than its adjacent neighbors (top, bottom, left, right), excluding any cells where the height is above a certain noise threshold.

To help guide the rover better, **if multiple valid local minima exist, choose the last one from the bottom right of the matrix.**

A local minimum is a cell strictly less than its up, down, left, and right neighbors (4 directions).

Ignore any cell or neighbor that is above the given noise threshold.

Input Array	Returned Array	Explanation

`matrix = [`
`[ 22, 21, 1000, 23],`
`[ 25, 17, 21, 1000],`
`[1000, 19, 18, 22],`
`[ 21, 22, 1000, 20]`
`]`
`noise_threshold = 100`
	Local minimum found at: (2, 2)	● (2, 2) has value 18.
		● Its neighbors: 21 (up), 1000 (down, ignored), 19 (left), 22 (right).
		● 18 is less than all valid neighbors and the bottom rightmost item

Python Notation	Java Notation

`def localMinima(matrix, threshold):`
`  # To DO`
	`public static void localMinima`
	`  (int[][] matrix, int threshold){`
	`  //To DO`
	`}`

Question 2: CO5 [8 Points]

You are asked to implement part of a **`Hashtable`** class where the keys are integers and the values are strings. The class has an array of Node type named **`ht`**, which is considered the hashtable. You have to write the class method **`insert`**, which should insert a new key-value pair in the hashtable.

The class method **`hash(key)`** method computes the hash index based on the key and returns the index of the hashtable. This method is already written for you. You shall use this while inserting a key-value pair.

While inserting, if there's any collision, you should follow the separate chaining approach; however, you should make sure that the nodes are added in ascending order of the keys. If there's any duplicate key, replace the previous value of that corresponding key.

Make sure that the time complexity of your code is O(n)

The Node class (non-dummy-headed singly type) for the hashtable is already implemented with key (int), val (String), and next (Node) variables. You do not need to write it.

Initial Hashtable **`ht`**	After executing `insert(8,"GOAT")`

`Index 0: (3, CAT) -> (4, EELS)`
`Index 1: (5, LION)`
`Index 2: (2, DOG) -> (9, CROW)`
`Index 3: (27, WOLF)`
`Index 4: (10, FISH) -> (11, OWL)`
	`Assume 8 is mapped at index 2.`
	`Index 0: (3, CAT) -> (4, EELS)`
	`Index 1: (5, LION)`
	`Index 2: (2, DOG) -> (8, GOAT) -> (9, CROW)`
	`Index 3: (27, WOLF)`
	`Index 4: (10, FISH) -> (11, OWL)`

Use the variable named "**ht**" to access the hashtable inside the method insert.

```
# Python notation

def hash(key):
    # use this function
    # already written

def  insert(key, value):
    # Write your code here
```

```
// Java notation

int hash(int key){
    // use this function
    // already written
}
void insert(int key, String value){
    // Write your code here
}
```

Question 3: CO3 [8 Points]

Suppose you are developing a personal banking app that tracks daily account activity. Each transaction recorded in the app represents either a deposit (a positive integer) or a withdrawal (a negative integer). These transactions are stored in a **Stack**, with the most recent transaction on top.

You need to implement a function named **depositSum()** that takes two arguments: a stack of transaction data and an integer representing the window size. The function should compute the **sum of only the deposit values** within each consecutive window of the given size and return the modified stack containing these summed values. If the last window contains fewer elements than the specified window size, it should still be processed using the remaining values. The most recent window's sum should appear at the top of the returned stack.

The Stack class is implemented for you. **Its public instance methods are push(), pop(), peek(), and isEmpty()**. The isEmpty() method returns a boolean indicating whether the stack is empty. The peek() and pop() methods return null or None if the stack is empty (i.e., underflow condition). All other members of the Stack class are private and should not be accessed directly. You can only use the public instance methods provided. No need to consider the overflow exception for the Stack class.

You cannot use any data structure other than the stack. You can create instances of the stack as many times as you want.

Stack Creation Example

Python	Java
`stack = Stack()`	`Stack stack = new Stack()`

Input Stack	Modified Stack	Explanation
stack = Top ➔ 12 -3 8 15 -7 -20 -6 -2 9 14 `window = 4`	stack = Top ➔ 35 0 23	Since the window size is given as 4, each window will have 4 values (except possibly the last one, if there are fewer remaining elements). Window 1: has [12, -3, 8, 15] Sum of deposits: 12 + 8 + 15 = 35 Window 2: has [-7, -20, -6, -2] Sum of deposits: 0 (Sum is 0 since no deposits in this window) Window 3: has [9, 14] (remaining 2 elements) Sum of deposits: 9 + 14 = 23 Modified stack means you must return the stack you received via the function parameter.

Python	Java

def **depositSum(st, window):**
    # To Do
	public static Stack **depositSum(Stack st, int window)**{
	    // To Do
	}

Question 4: CO2 [2 + 2 + 2 Points]
(Write the correct answers in the answer script.)

I. Given a 3D array index [1][2][3] from an array with dimensions 2×3×4, determine whether there exists a corresponding valid 2D array index [i][j] in a 2D array of dimensions 5×3 that maps to the same linear index. If such a 2D index exists, provide it. If not, explain why it is invalid.

II. In case of memory consumption, which one do you think performs better between a linked list and a hashtable using forward chaining as collision handling? Briefly justify your answer in terms of space complexity.

III. If the top pointer of the stack pointed to the first inserted element instead of the last inserted one, what would be the time complexity of the stack operations pop and peek in a linked list-based implementation? Explain.